03 - Implement Integration

In this exercise, you'll integrate your chatbot frontend with the backend API you built in the previous exercises. This will create a complete, working chatbot application where users can send messages and receive responses.

What You'll Learn

By the end of this exercise, you will have:

- Connected your frontend chatbot UI to the backend API
- Implemented real-time message sending and receiving
- Verified the complete end-to-end functionality
- Learned to debug network requests using browser developer tools
- Created a fully functional chatbot application

Step 1: Integrate Frontend with Backend

Now that you have both a frontend and backend, instruct Kiro to connect them together.

```
Update the frontend to send user messages to the backend API. When receiving a response from the backend, update the chat UI with the response.

The integration should:
- Send POST requests to the API Gateway endpoint when users submit messages
- Display the backend response (ACK + timestamp) in the chat interface
- Handle any errors gracefully
- Maintain the existing UI design and functionality

Please update the frontend code and redeploy it.
```

Integration Focus
This step transforms your chatbot from a static demo into a working application. The frontend will now communicate with your serverless backend in real-time.

Step 2: Test the Complete Application

Once Kiro has updated the frontend, please test the complete chatbot application by:

1. Opening the frontend URL
2. Sending a test message
3. Verifying that the backend responds with an ACK and timestamp
4. Checking that the response appears correctly in the chat UI

Expected Behavior

When you type a message like "Hello, chatbot!" in the UI, you should see:

1. Your message appears in the chat interface
2. A response from the backend appears below it
3. The response includes an acknowledgment and timestamp

Step 3: Verify Network Communication

Use your browser's developer tools to confirm the frontend is actually communicating with the backend.

Open Developer Tools

Chrome | Firefox | Safari

1. Right-click on the chatbot page
2. Select **Inspect** or press `F12`
3. Click the **Network** tab
4. Send a message in the chatbot
5. Look for the POST request to your API Gateway endpoint

What to Look For

In the Network tab, you should see:

- A POST request to your API Gateway URL
- Status code 200 (success)
- Response containing your ACK message and timestamp
- Request payload containing your chat message

Network Verification
Seeing the network requests confirms that your frontend and backend are properly integrated and communicating through the API Gateway.

Step 4: Validate End-to-End Functionality

Ask Kiro to help you verify everything is working correctly:

```
Please help me verify the complete chatbot functionality by:
1. Confirming the frontend sends messages to the correct API endpoint
2. Checking that the backend Lambda function receives and processes messages
3. Verifying the response appears correctly in the chat UI
4. Testing with a few different messages to ensure consistency

Also show me the CloudWatch logs to confirm the Lambda function is executing properly.
```

Expected Results

After completing this exercise, you should have:

- ✅ **Integrated Application**: Frontend and backend working together seamlessly
- ✅ **Real-time Communication**: Messages sent from UI reach the backend API
- ✅ **Proper Responses**: Backend responses appear correctly in the chat interface
- ✅ **Network Verification**: Developer tools show successful API requests
- ✅ **End-to-End Testing**: Complete chatbot functionality confirmed
- ✅ **Monitoring**: CloudWatch logs show Lambda function execution

Troubleshooting Common Issues

▶ **CORS Errors**

▶ **Network Request Fails**

▶ **Backend Not Responding**

▶ **Lambda Function Errors**

Architecture Overview

▶ **View Complete Application Architecture**

Next Steps

Congratulations! You now have a complete, working chatbot application. The frontend and backend are fully integrated, and users can have real conversations with your bot. Now it's time to add AI intelligence to make your chatbot truly smart.

Workshop Progress


```
COMPLETED        COMPLETED        COMPLETED
```

Let's Enhance Chatbot Security

Previous | Next